

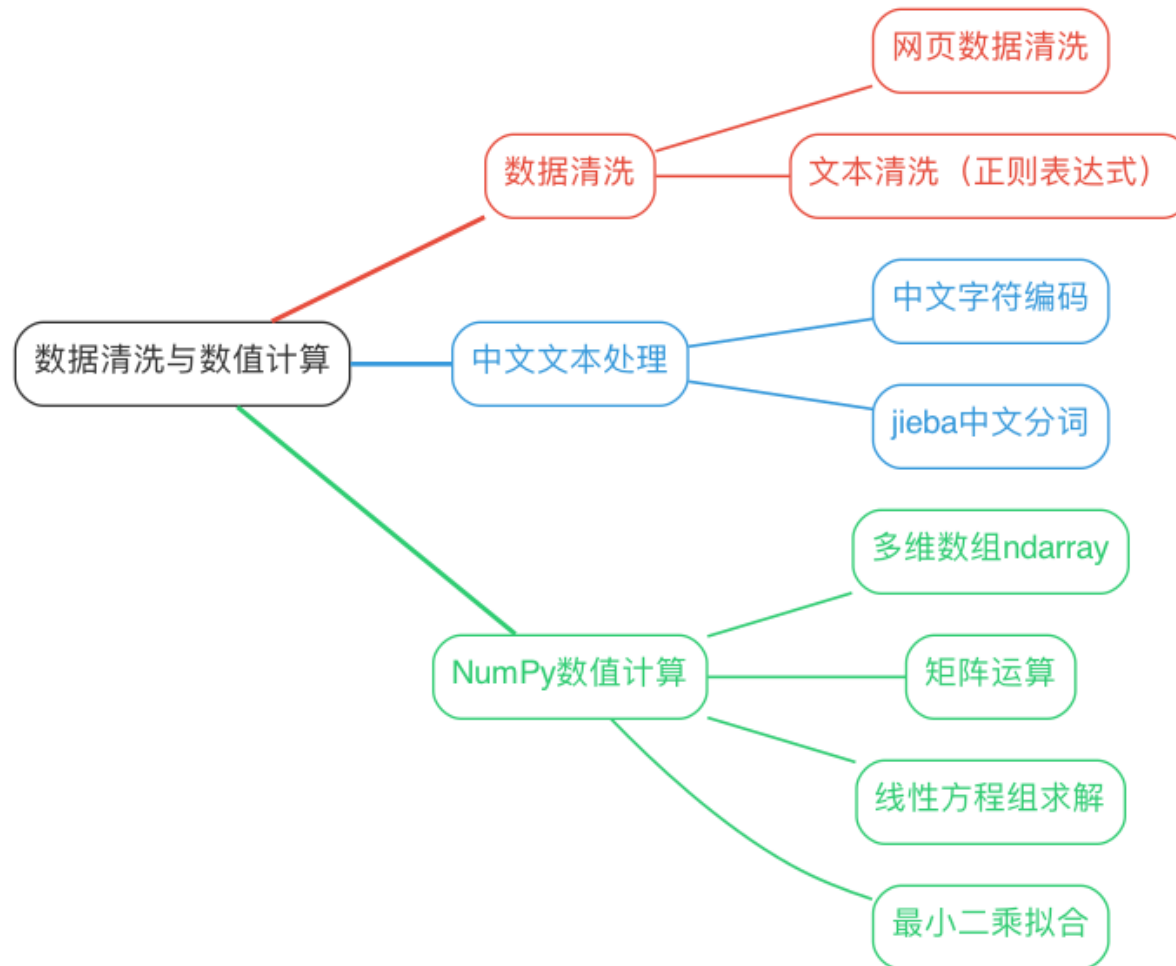


《实用Python编程》

第7讲：数据清洗与数值计算

实用Python编程 教研组

提纲





网页数据清洗

• 目标网页（教育部985高校名单）

中华人民共和国教育部
Ministry of Education of the People's Republic of China

当前位置：首页 > 公开

信息名称： “985工程”学校名单
信息索引： 360A22-07-2006-0655-1
生成日期： 2006-12-06
发文机构： 教育部学位管理与研究生教育司

发文字号：
内容概述： 经教育部、财政部批准建设的“985工程”高等学校名单

信息类别： 高等教育

“985工程”学校名单

北京大学	中国人民大学	清华大学
北京航空航天大学	北京理工大学	中国农业大学
北京师范大学	中央民族大学	南开大学
天津大学	大连理工大学	东北大学
吉林大学	哈尔滨工业大学	复旦大学
同济大学	上海交通大学	华东师范大学
南京大学	东南大学	浙江大学
中国科学技术大学	厦门大学	山东大学
中国海洋大学	武汉大学	华中科技大学
湖南大学	中南大学	国防科学技术大学
中山大学	华南理工大学	四川大学
电子科技大学	重庆大学	西安交通大学
西北工业大学	西北农林科技大学	兰州大学

利用第3章的PageDownloader抓取

```
1 import os, pandas as pd
2 from p3lib.ch3 import url_to_filename, PageDownloader #从自定义库p3lib中导入第3章网页抓取相关的函数与类
3
4 url = 'http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html' # 目标网址
5 html_file = os.path.join('html_pages', url_to_filename(url) + ".html") # 本地文件路径
6 print(html_file)
7
8 if os.path.exists(html_file) is False:
9     page = PageDownloader()
10    page.process(url)
11 assert(os.path.exists(html_file))
```

http://www.moe.gov.cn/srcsite/A22/s7065/200612/t20061206_128833.html



网页数据清洗

• 如何从抓取的HTML文件中提取目标数据？

中华人民共和国教育部
Ministry of Education of the People's Republic of China

当前位置：首页 > 公开

信息名称： “985工程”学校名单
信息索引： 360A22-07-2006-0655-1
生成日期： 2006-12-06
发文机构： 教育部学位管理与研究生教育司

发文字号：
内容概述： 经教育部、财政部批准建设的“985工程”高等学校名单

信息类别： 高等教育

“985工程”学校名单

北京大学	中国人民大学	清华大学
北京航空航天大学	北京理工大学	中国农业大学
北京师范大学	中央民族大学	南开大学
天津大学	大连理工大学	东北大学
吉林大学	哈尔滨工业大学	复旦大学
同济大学	上海交通大学	华东师范大学
南京大学	东南大学	浙江大学
中国科学技术大学	厦门大学	山东大学
中国海洋大学	武汉大学	华中科技大学
湖南大学	中南大学	国防科学技术大学
中山大学	华南理工大学	四川大学
电子科技大学	重庆大学	西安交通大学
西北工业大学	西北农林科技大学	兰州大学

在网页空白处单击鼠标右键，
选中“显示网页源代码”

```
160 <table class="MsoNormalTable" style="width: 100%; border-collapse: collapse" cellspacing="0" cellpadding="0" width="100%" border="0">
161 <tbody>
162 <tr style="height: 22.7pt">
163 <td style="border-top: windowtext 1pt solid; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.44%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: windowtext 1pt solid; padding-right: 5.4pt" width="33%">
164 <p class="MsoNormal" style="text-align: center" align="center"><a name="OLE_LINK2"></a><a name="OLE_LINK4"></a><span style="font-family: 宋体">北京大学</span></p>
165 </td>
166 <td style="border-top: windowtext 1pt solid; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.14%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
167 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">中国人民大学</span></p>
168 </td>
169 <td style="border-top: windowtext 1pt solid; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.42%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
170 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">清华大学</span></p>
171 </td>
172 </tr>
173 <tr style="height: 22.7pt">
174 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.44%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: windowtext 1pt solid; padding-right: 5.4pt" width="33%">
175 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">北京航空航天大学</span></p>
176 </td>
177 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.14%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
178 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">北京理工大学</span></p>
179 </td>
180 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.42%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
181 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">中国农业大学</span></p>
182 </td>
183 </tr>
184 <tr style="height: 22.7pt">
185 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.44%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: windowtext 1pt solid; padding-right: 5.4pt" width="33%">
186 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">北京师范大学</span></p>
187 </td>
188 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.14%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
189 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">中央民族大学</span></p>
190 </td>
191 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.42%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
192 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">南开大学</span></p>
193 </td>
194 </tr>
195 <tr style="height: 22.7pt">
196 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.44%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: windowtext 1pt solid; padding-right: 5.4pt" width="33%">
197 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">天津大学</span></p>
198 </td>
199 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.14%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
200 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">上海交通大学</span></p>
201 </td>
202 <td style="border-top: medium none; height: 22.7pt; border-right: windowtext 1pt solid; width: 33.42%; border-bottom: windowtext 1pt solid; padding-bottom: 0cm; padding-top: 0cm; padding-left: 5.4pt; border-left: medium none; padding-right: 5.4pt" width="33%">
203 <p class="MsoNormal" style="text-align: center" align="center"><span style="font-family: 宋体">复旦大学</span></p>
204 </td>
205 </tr>
```



网页数据清洗

• 方案1：利用BeautifulSoup提取目标数据

分析源代码后的发现：

目标数据位于网页第2个表格

目标字段被标签包裹

```
1 from bs4 import BeautifulSoup
2
3 html_file = 'html_pages/aHR0cDovL3d3dy5tb2UuZ292LmNuL3NyY3NpdGUvQTIyL3M3MDY1LzIwMDYxMi90MjAwNjEyMDZfMTI4ODMzMmh0bWw.html'
4 html_txt = open(html_file).read()
5 soup = BeautifulSoup(html_txt)
6 table = soup.find_all("table")[1] # 分析源代码, 可知目标数据位于网页第二个表格中
7 tags = table.find_all('span')
8 data = [tag.text for tag in tags]
9 print(data)
```

```
['北京大学', '北京大学', '中国人民大学', '清华大学', '北京航空航天大学', '北京理工大学', '中国农业大学', '北京师范大学', '中央民族大学', '南开大学', '天津大学', '大连理工大学', '东北大学', '吉林大学', '哈尔滨工业大学', '复旦大学', '同济大学', '上海交通大学', '华东师范大学', '南京大学', '东南大学', '浙江大学', '中国科学技术大学', '厦门大学', '山东大学', '中国海洋大学', '武汉大学', '华中科技大学', '湖南大学', '中南大学', '国防科学技术大学', '中山大学', '华南理工大学', '四川大学', '电子科技大学', '重庆大学', '西安交通大学', '西北工业大学', '西北农林科技大学', '兰州大学']
```

网页数据清洗

- 方案2：调用Pandas（4.4节）的`read_html`函数

```
tables = pd.read_html(html_file) # 使用pandas读取HTML中的所有表格
tab = tables[1] # 985高校名单在第2个表格中
colleges = tab.values.flatten() # 将所有值展平为一维数组
```

tab是一个DataFrame对象

	0	1	2
0	北京大学	中国人民大学	清华大学
1	北京航空航天大学	北京理工大学	中国农业大学
2	北京师范大学	中央民族大学	南开大学
3	天津大学	大连理工大学	东北大学
4	吉林大学	哈尔滨工业大学	复旦大学
5	同济大学	上海交通大学	华东师范大学
6	南京大学	东南大学	浙江大学
7	中国科学技术大学	厦门大学	山东大学
8	中国海洋大学	武汉大学	华中科技大学
9	湖南大学	中南大学	国防科学技术大学
10	中山大学	华南理工大学	四川大学
11	电子科技大学	重庆大学	西安交通大学
12	西北工业大学	西北农林科技大学	兰州大学

展平



array(['北京大学', '中国人民大学', '清华大学', '北京航空航天大学', '北京理工大学', '中国农业大学', '北京师范大学',
'中央民族大学', '南开大学', '天津大学', '大连理工大学', '东北大学', '吉林大学', '哈尔滨工业大学',
'复旦大学', '同济大学', '上海交通大学', '华东师范大学', '南京大学', '东南大学', '浙江大学',
'中国科学技术大学', '厦门大学', '山东大学', '中国海洋大学', '武汉大学', '华中科技大学', '湖南大学',
'中南大学', '国防科学技术大学', '中山大学', '华南理工大学', '四川大学', '电子科技大学', '重庆大学',
'西安交通大学', '西北工业大学', '西北农林科技大学', '兰州大学'], dtype=object)



文本清洗

正则表达式 (Regular Expression)

- 基于模式的字符串匹配
 - 模式串以**字母r**为前缀
- 加强版的ctrl + F

理解模式串“\d{4}”与“^\d{4}\D”的区别

- ^ 表示开始匹配的位置需在主串开头
- \D 表示匹配到的子串后面应接一个非数字字符

当指定的模式串不存在时, re.search()返回None

若m不为None, m.group() 返回**匹配到**的字符串

```
1 import re
2
3 pattern = r"\d{4}"
4 pattern_2 = r"^\d{4}\D"
5 text_list = """2021年安徽省录取分数线
6 2023年江西省录取分数线
7 2022年内蒙古自治区录取分数线
8 2022年上海市录取分数线
9 202年内蒙古自治区录取分数线
10 20229年内蒙古自治区录取分数线""".split()
11
12 for text in text_list:
13     m = re.search(pattern, text)
14     if m: # 等同于 if m is not None:
15         print("匹配结果", m.group())
16     else:
17         print("匹配失败")
```

匹配结果 2021
匹配结果 2023
匹配结果 2022
匹配结果 2022
匹配失败
匹配结果 2022

文本清洗

• 正则表达式语法

从指定文本中提取**年份**和**省份**这两项数据

'2021年安徽省录取分数线'

```
m = re.search(r'(\d+)年(\D+)录取分数线', '2021年安徽省录取分数线')
m.groups()
```

('2021', '安徽省')

修改模式串
'年' → '年?'

'2023安徽省录取分数线'

```
m = re.search(r'(\d+)年?(\D+)录取分数线', '2023安徽省录取分数线')
m.groups()
```

('2023', '安徽省')

表 4.1: 正则表达式常用符号及对应表达式语法

符号	功能	示例
指代:		
\d	匹配一个数字字符	<code>r"a\d b"</code> , 匹配 a0b, a1b, ..., a9b
\D	匹配一个非数字字符	<code>r"a\D"</code> , 不匹配 a0, a1, ..., a9
\w	匹配一个字母、数字或下划线	<code>r"a\w"</code> , 匹配 aa, a0, a_ 等
\s	匹配一个空格或制表符	<code>r"a\s b"</code> , 匹配 "a b"
.	通配符	<code>r"a.b"</code> , 匹配 a0b, aab, a b 等
\.	匹配点号	<code>r"a\.b"</code> , 匹配 a.b
\-	匹配连接符号	<code>r"a-b"</code> , 匹配 a-b
[]	限定范围	<code>r"[12]"</code> , 仅匹配 1 或 2
[^]	反向限定	<code>r"[^12]"</code> , 匹配除 1 和 2 之外的其他字符
	多选	<code>r"(apple orange)"</code> , 匹配 apple 或 banana
计数:		
+	匹配 1 次或更多	<code>r"a+"</code> , 匹配 a, aa, aaa, ...
*	匹配 0 次或更多	<code>r"ba*"</code> , 匹配 b, ba, baa, baaa, ...
?	匹配 0 次或 1 次	<code>r"colou?r"</code> , 匹配 color 或 colour
{m}	匹配 m 次	<code>r"\d{4}"</code> , 匹配 4 位数字
{m,n}	匹配 m 至 n 次	<code>r"\d{4,6}"</code> , 匹配长度不低于 4、不超过 6 的数字串
限位:		
^	匹配字符串的开头	<code>r"^Hello"</code> , 匹配以 Hello 开头的文本
\$	匹配字符串的结尾	<code>r"world\$"</code> , 匹配以 world 结尾的文本
()	分组捕获匹配内容	<code>r"(\d{3,4})\-(\d{7,8})"</code> , 分组匹配固话的区号和用户号码



课堂练习-4-1-1

- 霍比特人有声书文件名规范化

文本文件 `pybook-data/ch4/hobbit-playlist.txt` 中包含了一系列不规则的视频路径。请将这些旧路径批量重命名为规则的新文件名，并按照章节顺序打印输出

- 输入样例（旧路径）

```
The-Hobbit/【很美一版】The Hobbit 霍比特人有声书 read by Bluefax (P14. 14 Fire And Water [2019 Edition].mp4  
The-Hobbit/【很美一版】The Hobbit 霍比特人有声书 read by Bluefax (P2. 02 Roast Mutton [2019 Edition]).mp4
```

- 输出样例（新文件名，按章节顺序排列）

```
The-Hobbit-02-Roast-Mutton.mp4  
The-Hobbit-14-Fire-And-Water.mp4
```



中文文本处理与分析

- 含汉字在内的所有字符在计算机中均以整数编码
- 内置函数 `ord`
 - 输入：一个字符（也就是长度为1的字符串）
 - 返回：该字符在Unicode（或ASCII）标准中所对应的整数码点

```
1 # 英文字符 (对应经典的 ASCII 码)
2 print(ord('A')) # 输出: 65
3 print(ord('a')) # 输出: 97
4 print(ord('0')) # 输出: 48
5
6 # 中文字符 (对应现代的 Unicode 码)
7 print(ord('中')) # 输出: 20013
8 print(ord('国')) # 输出: 22269
```

内置函数 `chr`: `ord` 的反向操作

```
1 chr(20013), chr(22269), chr(128512)
('中', '国', '😄')
```



中文字符编码

- 常用汉字的Unicode整数码点的范围是 19,968~40,959
- 为便于记录，整数码点一般用十六进制表示（即逢16进1）
 - 十六进制数以0x作为前缀，字符a~f对应10~15
 - 0x10 对应 16, 0x1f 对应 31
- 用内置函数hex可将整数转为以字符串表示的十六进制数
 - hex(31) → '0x1f', hex(19968) → '0x4e00', hex(40959) → '0x9fff'
- 用int可将十六进制字符串变回原来的整数
 - int('0x1f', 16)返回整数31 # 注意在调用时额外指定了进制参数base为16

中文字符编码

- 根据Unicode编码过滤掉非中文字符

```
1 def is_cn_char(c): # 自定义函数, 用于判断输入字符是否为汉字
2     code = ord(c)
3     return code >= 0x4e00 and code <= 0x9fff
4
5 text = "2024年, 贵州农产品加工转化率达66%。"
6 chinese = [] # 创建一个空的列表, 用于收集中文字符
7 for c in text: # 通过for循环遍历字符串text中的每个字符
8     if is_cn_char(c): # 若为中文字符, 则添加到列表中
9         chinese.append(c)
10 print(chinese)
```

['年', '贵', '州', '农', '产', '品', '加', '工', '转', '化', '率', '达']



课堂练习-4-1-2

- 现有一名特工人员，其代码为“左弃圆”。该代号由其真实名字加密得到。加密规则为每个字的整数码点加9之后再经chr变回字符。请编写一个解码函数`decode`，获得该特工的真名。



中文分词

- 用jieba进行中文分词
 - 命令行安装指令 `pip install jieba`
 - 导入方式 `import jieba`

```
1 import jieba
2 import jieba.posseg as pseg # 用pseg作为jieba.posseg的别名
3
4 text = '特朗普私下承认：共和党要守住参议院多数席位非常困难'
5 words = jieba.cut(text)
6 print("/".join(words)) # 特朗普/私下/承认/: /共和党/要/守住/参议院/多数/席位/非常/困难
7
8 words = pseg.cut(text) # 分词的同时标记词性, 返回单词-词性配对序列
9 res = []
10 for (word, tag) in words:
11     res.append(f"{word}/{tag}") # 用f-string方式拼接word与tag
12
13 print("/".join(res))
```

特朗普/私下/承认/: /共和党/要/守住/参议院/多数/席位/非常/困难

特朗普/nr/私下/n/承认/v/: /x/共和党/nt/要/v/守住/v/参议院/n/多数/m/席位/n/非常/d/困难/an

paddle模式词性和专名类别标签集合如下表，其中词性标签 24 个（小写字母），专名类

标签	含义	标签	含义	标签	含义	标签	含义
n	普通名词	f	方位名词	s	处所名词	t	时间
nr	人名	ns	地名	nt	机构名	nw	作品名
nz	其他专名	v	普通动词	vd	动副词	vn	名动词
a	形容词	ad	副形词	an	名形词	d	副词
m	数量词	q	量词	r	代词	p	介词
c	连词	u	助词	xc	其他虚词	w	标点符号
PER	人名	LOC	地名	ORG	机构名	TIME	时间

<https://github.com/fxsjy/jieba>



中文分词

- 通过`add_word`往jieba词典中添加自定义词汇
 - 让分词器能够将这些新词识别为一个整体，而不是被拆分开来
 - 用于处理未被jieba内置词典涵盖的专业术语、人名、地名、网络新词等
 - 函数参数
 - word: 要添加的自定义词，必填
 - freq: 词频，可选。值越大，该词在分词时被单独切分出来的优先级就越高
 - tag: 词性标注，可选

```
1 text = "乒乓球拍卖完了"
2 words = pseg.cut(text)
3 print("/".join([word for (word, tag) in words])) # 乒乓球/拍卖完/了
4
5 jieba.add_word("乒乓球拍", freq=1000, tag="n") # 将自定义词汇加入到词典中，词汇权重由freq参数指定
6 words = pseg.cut(text)
7 print("/".join([word for (word, tag) in words])) # 乒乓球拍/卖完/了
```

乒乓球/拍卖/完/了

乒乓球拍/卖完/了

中文文本分析

• 预处理 → 分词 → 词频统计 → 可视化

非常感谢。这是巨大的荣幸。今天是非常精彩的一天。尤其是，我要感谢习近平主席，我的朋友给予我们如此盛大的欢迎。这确实是一场极其隆重的欢迎仪式，无与伦比，也感谢他如此盛情地接待我们，为了这次具有历史意义的国事访问。今天，我们与中方代表团举行了非常积极且富有成果的对话与会谈。而今晚，又是一次值得珍惜的机会，让朋友之间讨论我们今天谈到的一些议题——这些都对美国和中国有利。与你在一起，非常荣幸。

美国人民与中国人民之间的关系，可以追溯到美国建国时期。美国首任驻华领事塞缪尔·肖 (Samuel Shaw) 于1784年乘坐首艘美国商船抵达中国海岸。中国商人当时给美国人起了一个名字，称他们为“新来的人” (the “new people”)。两个半世纪之后，这一最初的联系已经发展成为世界历史上最具有重要意义的双边关系之一。

原始文本

非常感谢/。/这是/巨大/的/荣幸/。/今天/是/非常/精彩/的/一天/。/尤其是/，/我要/感谢/习近平/主席/，/我/的/朋友/，/给予/我们/如此/盛大/的/欢迎/。/这/确实/是/一场/极其/隆重/的/欢迎仪式/，/无与伦比/，/也/感谢/他/如此/盛情/地/接待/我们/，/为了/这次/具有/历史/意义/的/国事访问/。/今天/，/我们/与/中方/代表团/举行/了/非常/积极/且/富有成果/的/对话/与/会谈/。/而/今晚/，/又/是/一次/值得/珍惜/的/机会/，/让/朋友/之间/讨论/我们/今天/谈到/的/一些/议题/——/这些/都/对/美国/和/中国/有利/。/与/你/在/一起/，/非常/荣幸/。/ /

/美国/人民/与/中国/人民/之间/的/关系/, /可以/追溯到/美国/建国/时期/. /美国/首任/驻华/领事/塞缪尔-/肖/(/Samuel/ /Shaw/)于1784/年/乘坐/首艘/美国/商船/抵达/中国/海岸/. /中国/商人/当时/给/美国/人起/了/一个/名字/, /称/他们/为/"新来/的/人"/(/the/ /"new/ /people"/). /两个/半世纪/之后/, /这一/最初/的/联系/已经/发展/成为/世界/历史/上/最具/重要/意义/的/双边关系/之一/。

分词后





基于NumPy的数值数据操作与计算

- NumPy是目前Python科学计算中最重要的基础库
 - 命令行安装指令 `pip install numpy`
 - 导入方式 `import numpy as np`



<https://numpy.org/>

检查NumPy是否就绪

- 在命令行中执行指令 `python -c "import numpy as np; print(np.__version__)"`
- 在Notebook中执行语句

```
1 import numpy as np
2 np.__version__
```

'2.4.1'



数值计算案例

奶茶店饮品价格计算

- 某奶茶店销售三种饮品：珍珠奶茶、烧仙草、杨枝甘露。
- 销售数据如下：
 - 周一：卖出珍珠奶茶 20 杯、烧仙草 15 杯、杨枝甘露 10 杯，总收入 410 元
 - 周二：卖出珍珠奶茶 15 杯、烧仙草 10 杯、杨枝甘露 20 杯，总收入 410 元
 - 周三：卖出珍珠奶茶 10 杯、烧仙草 20 杯、杨枝甘露 15 杯，总收入 395 元
- 求三种饮品的单价（单位：元/杯）

数值计算案例

奶茶店饮品价格计算

- 分别用 x , y , z 表示珍珠奶茶、烧仙草、杨枝甘露的单价
- 该任务就是要求解三元一次方程组

$$\begin{cases} 20x + 15y + 10z = 410 \\ 15x + 10y + 20z = 410 \\ 10x + 20y + 15z = 395 \end{cases}$$

$$\underbrace{\begin{bmatrix} 20 & 15 & 10 \\ 15 & 10 & 20 \\ 10 & 20 & 15 \end{bmatrix}}_{\text{系数矩阵 } A} \underbrace{\begin{bmatrix} x \\ y \\ z \end{bmatrix}}_{\text{未知数向量}} = \underbrace{\begin{bmatrix} 410 \\ 410 \\ 395 \end{bmatrix}}_{\text{常数向量 } b}$$

如何表示系数矩阵 A 和常数向量 b ?



认识ndarray

- 多维数组 (ndarray) 是NumPy的核心数据类型
 - 可看成是元素必须是数值型的、支持数值计算的特殊列表 (list)

常见的创建ndarray对象的方式

```
1 import numpy as np # 导入NumPy并以np作为别名来使用
2
3 zero_0 = np.array([]) # 一个空数组
4 zero_1 = np.array([0, 0, 0, 0]) # 列表转数组
5 zero_2 = np.array((0, 0, 0, 0)) # 元组转数组
6 zero_3 = np.zeros(5) # 长度为5的一维全0数组
7 zero_4 = np.zeros((5,3)) # 5行3列的二维全0数组
8 ones = np.ones(5) # 长度为5的一维全1数组, 即array([1., 1., 1., 1., 1.])
9 seq = np.arange(0, 10, 2) # array([0, 2, 4, 6, 8])
10 identity = np.eye(3) # 3x3的单位矩阵
11 arr = np.array([[1,2,3], [4,5,6]]) # 2行3列的二维数组
```

表 4.2: 多维数组ndarray的常用属性。变量 *arr* 由示例代码4.11给出

属性名	含义	示例代码	示例属性值
ndim	数组的维度数	arr.ndim	2
shape	以元组形式返回的数组各维度大小	arr.shape	(2,3)
size	数组元素的总个数	arr.size	6
dtype	数组元素的数据类型	arr.dtype	int64

认识ndarray

- 对比ndarray与list

```
1 a = [1, 1, 1]
2 b = [2, 2, 2]
3 array_a = np.array(a)
4 array_b = np.array(b)
5
6 print(a + b) # [1, 1, 1, 2, 2, 2]
7 print(array_a + array_b) # [3 3 3]
8
9 np.array([1,2]) + np.array([0,0,3])
```

list是一个容器，两个list对象相加会拼接扩容
ndarray是向量，两个ndarray对象相加执行向量加法

向量尺寸不一致，报错

操作ndarray

操作类型

- 整体: `astype`
- 变形: `reshape`
- 拼接: `vstack/hstack`
- 查找: `argmin/argmax`
- 选择: `np.where(a>3)`
- 切片: `b[:,0]`, `b[0,:]`

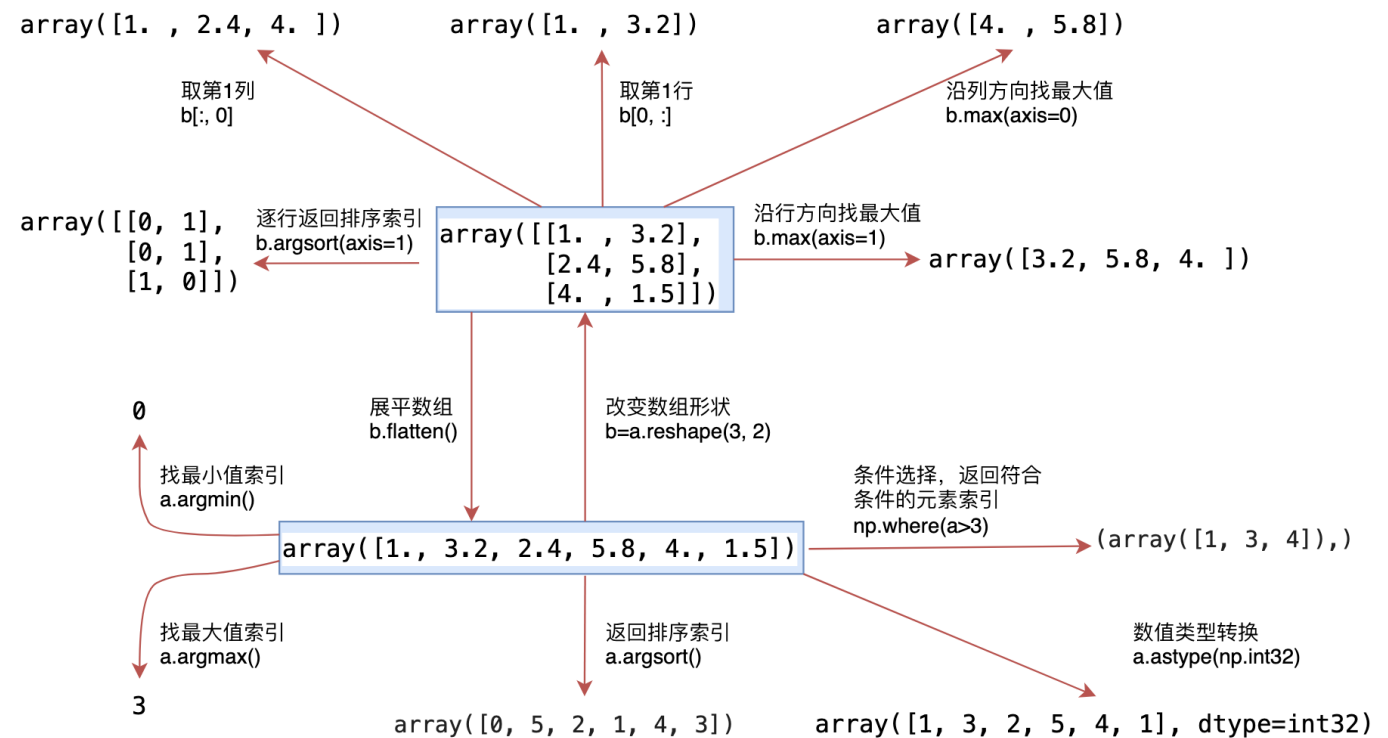


图 4.3: 多维数组 `ndarray` 常用操作示例。变量 `a` 和 `b` 分别指向图中的一维和二维数组

矩阵运算

- 数值分析以矩阵为基础
- 一个 $m \times n$ 的矩阵可用一个 m 行、 n 列的二维数组表示

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[1, 0], [0, 1]]) # 单位矩阵, 简称单位阵
3 C = A * B # 点乘 - 对应元素相乘
4 print("A*B")
5 print(C)
6 D = A @ B # 矩阵相乘 - 因B是单位阵, 所以D等于A
7 print("A@B")
8 print(D)
```

A*B

```
[[1 0]
 [0 4]]
```

A@B

```
[[1 2]
 [3 4]]
```

线性方程组求解

- 回到之前的案例
 - 分别用两个ndarray表示系数矩阵与常数向量
 - 再调用NumPy的线性代数模块np.linalg.solve求解方程组

$$\underbrace{\begin{bmatrix} 20 & 15 & 10 \\ 15 & 10 & 20 \\ 10 & 20 & 15 \end{bmatrix}}_{\text{系数矩阵}A} \underbrace{\begin{bmatrix} x \\ y \\ z \end{bmatrix}}_{\text{常数向量}b} = \underbrace{\begin{bmatrix} 410 \\ 410 \\ 395 \end{bmatrix}}_{\text{常数向量}b}$$

```
1 A = np.array([[20,15,10], [15,10,20], [10,20,15]])
2 b = np.array([410, 410, 395])
3
4 solution = np.linalg.solve(A, B)
5
6 products = "珍珠奶茶 烧仙草 杨枝甘露".split()
7 print("--- 求解结果 ---")
8 for i,pro in enumerate(products):
9     print(f"{pro}的单价是 {solution[i]:.2f}")
```

```
--- 求解结果 ---
珍珠奶茶的单价是 10.00
烧仙草的单价是 8.00
杨枝甘露的单价是 9.00
```

小数点后
保留两位



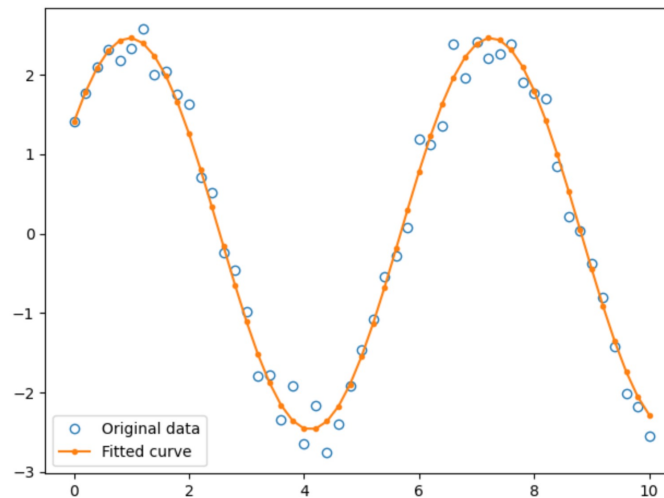
课堂练习-4-1-3

- 验证solution是否正确, 即 $A \cdot \text{solution}$ 是否等于 b

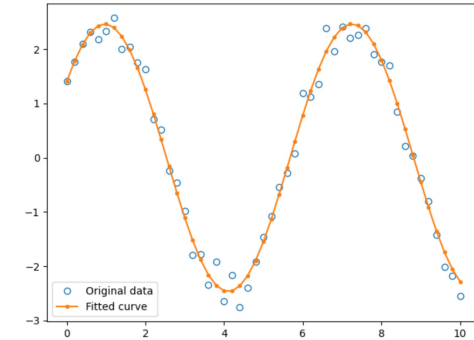
```
1 A = np.array([[20,15,10], [15,10,20], [10,20,15]])
2 b = np.array([410, 410, 395])
3
4 solution = np.linalg.solve(A, B)
5
6 products = "珍珠奶茶 烧仙草 杨枝甘露".split()
7 print("---- 求解结果 ----")
8 for i,pro in enumerate(products):
9     print(f"{pro}的单价是 {solution[i]:.2f}")
```

最小二乘拟合

- 数据拟合 (Fitting) 就是找到一个数学模型 (函数), 使其能够最贴合已有的观测数据点
- 最小二乘法 (Least Squares Method) 是最常用的拟合方法之一
 - 目标是使拟合出的数据与实际数据之间误差的平方和最小



最小二乘拟合



- 观测数据来自一个周期性的正弦波
 - $f(x) = a \cdot \sin(x) + b \cdot \cos(x)$ # 参数 a, b 待估计
- 由于观测误差的存在, 实际获得的数据 (用 y 表示) 是在 $f(x)$ 的基础上叠加了随机噪声 ϵ , 即 $y = f(x) + \epsilon$
- 给定 n 个观测数据点 $\{(x_1, y_1), \dots, (x_n, y_n)\}$, 理想情况下, 我们希望找到一对 (a, b) , 使得数据能完美拟合:

$$\begin{cases} a \cdot \sin(x_1) + b \cdot \cos(x_1) & = y_1 \\ \dots & \\ a \cdot \sin(x_n) + b \cdot \cos(x_n) & = y_n \end{cases}$$

最小二乘拟合

- 将方程组表示为矩阵形式

$$\begin{cases} a \cdot \sin(x_1) + b \cdot \cos(x_1) = y_1 \\ \dots \\ a \cdot \sin(x_n) + b \cdot \cos(x_n) = y_n \end{cases} \quad \Rightarrow \quad \underbrace{\begin{bmatrix} \sin(x_1) & \cos(x_1) \\ \dots & \dots \\ \sin(x_n) & \cos(x_n) \end{bmatrix}}_{\text{设计矩阵}} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\text{目标向量}} = \underbrace{\begin{bmatrix} y_1 \\ \dots \\ y_n \end{bmatrix}}_{\text{目标向量}}$$

- 有了设计矩阵和目标向量，就可调用NumPy中的最小二乘法函数 `np.linalg.lstsq` 求解 (a, b)

最小二乘拟合

- 完整代码

```
1 # 1. 合成数据:  $y = 2\sin(x) + 1.5\cos(x) +$  高斯噪声 (均值为0, 标准差为0.2)
2 x = np.linspace(0, 10, 51)
3 y = 2 * np.sin(x) + 1.5 * np.cos(x) + np.random.normal(0, 0.2, len(x))
4
5 # 2. 构造“设计矩阵” A, 第一列是  $\sin(x)$ , 第二列是  $\cos(x)$ 
6 A = np.column_stack([np.sin(x), np.cos(x)])
7
8 # 3. 求解  $A * [a, b].T = y$ , 返回值第一个是系数向量
9 coeffs, residuals, rank, s = np.linalg.lstsq(A, y, rcond=None)
10 a, b = coeffs
11
12 print(f"拟合出的系数: a={a:.2f}, b={b:.2f}")
```

拟合出的系数: a=2.01, b=1.53



课堂练习4-1-4

智能手机耗电量预测

- 某手机厂商的研发部门要建立一个电池续航预测模型。他们收集了不同用户在一天内的手机使用数据，包含三项指标

- 屏幕亮屏时间 st (单位: 小时)
- 大型游戏运行时间 gt (单位: 小时)
- 一天结束时消耗的电池电量百分比 y

- 假设电量消耗满足以下线性组合关系: $y=w_0+w_1*st+w_2*gt$

其中 w_0 是手机待机基础功耗, w_1 和 w_2 分别是亮屏和游戏的每小时耗电系数

- 请基于右侧的样本数据, 求解并打印 w_0 、 w_1 、 w_2 。并预测一个“亮屏 4 小时, 玩游戏 1.5 小时”的新用户会消耗多少电量?

```
# 样本数据: [亮屏时间, 游戏时间]
X_raw = np.array([
    [2.0, 0.0], # 用户 1: 亮屏 2 小时, 没玩游戏
    [4.0, 0.5], # 用户 2: 亮屏 4 小时, 玩游戏 0.5 小时
    [3.0, 1.0], # 用户 3: 亮屏 3 小时, 玩游戏 1.0 小时
    [6.0, 0.0], # 用户 4: 亮屏 6 小时, 没玩游戏
    [5.0, 2.0]  # 用户 5: 亮屏 5 小时, 玩游戏 2.0 小时
])

# 实际消耗的电量百分比 (%)
y = np.array([24.0, 47.0, 51.0, 56.0, 93.0])
```